

SNP-AV1: Stacked Neural Pruners for the 4K Intra-Frame Partition Search in AV1

Pablo Rosa

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
pablo.rosa@inf.ufpel.edu.br

Daniel Palomino

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
dpalomino@inf.ufpel.edu.br

Marcelo Porto

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
porto@inf.ufpel.edu.br

Luciano Agostini

Video Technology Research Group (ViTech)
Federal University of Pelotas
Pelotas, Brazil
agostini@inf.ufpel.edu.br

Abstract—The AOMedia Video 1 (AV1) is an open and royalty-free video codec whose intra-frame coding relies on a recursive partition search evaluating ten partition shapes per node. This search is exhaustive by construction and dominates the intra-frame encoding time at the highest quality configuration. This paper presents the Stacked Neural Pruners for AV1 (SNP-AV1), a machine-learning solution applying two neural networks at two distinct points of that search. The first reads thirty-six attributes of low-cost causal context and decides, before any rate-distortion evaluation is paid, how much of the subtree deserves to be opened. The second adds the measured rate-distortion cost of the undivided candidate and decides whether the AB and 4-way shapes are evaluated at all. Experimental results show an average time savings of 18.74% with a BD-BR of 0.59%, reaching 32.16% and 1.42% under a stronger calibration. SNP-AV1 does not dominate the native speed presets, which reconfigure the whole encoder, but it delivers operating points the preset ladder leaves uncovered.

Index Terms—Video coding, AV1, computational effort reduction, intra-frame block partitioning, machine learning, neural networks.

I. INTRODUCTION

Video is the dominant workload of the Internet, and the share of users watching online video every month has kept growing over the last years [1]. Alongside the standards defined by the international standardization bodies, such as the High Efficiency Video Coding (HEVC) [2] and the Versatile Video Coding (VVC) [3] produced by the Motion Picture Expert Group (ISO/IEC MPEG) and by the Video Coding Experts Group (ITU-T VCEG), the Alliance for Open Media (AOMedia) maintains the AOMedia Video 1 (AV1) [4] as an open and royalty-free alternative.

AV1 follows the state-of-the-art flow of hybrid video encoders, splitting the process into intra-frame prediction, inter-frame prediction, transforms, quantization, in-loop filters and entropy coding, and introduced new techniques in each of these steps [5], [6]. This coding efficiency comes at the price of computational effort: AV1 was shown to be superior to

HEVC under certain conditions [7], but encoding videos of the same bitrate and resolution took twice as long [8], a problem aggravated in 4K content.

One of the largest individual costs of AV1 intra-frame coding is the block partitioning decision. Every superblock is submitted to a recursive search that, at each node of the partition tree, evaluates the rate-distortion (RD) cost [9] of ten candidate shapes and keeps the cheapest one, descending recursively whenever the quad-tree split wins. It is exhaustive by construction, since the encoder only knows which shape is best after actually coding the block under each of them, and in the reference software libaom [10] this regime is what the highest quality preset, `cpu-used=0`, delivers.

There are a few works in the literature targeting the computational effort of AV1, and none of them attacks this search. The works [11]–[13] act on the intra prediction mode decision loop, while [14], [15] are hardware solutions for the AV1 intra prediction, whose target is the circuit and not the encoder search, and [16] applies decision trees to the inter-prediction. Learning-based partitioning solutions do exist for VVC [17]–[19] and can be adapted to the AV1 coding tools, but they were designed for the quad-tree with nested multi-type tree and evaluated on another encoder. In none of these cases is a direct and fair comparison possible, and none explores learned pruning of the AV1 intra-frame partition search itself.

This paper presents SNP-AV1, which is not another speed preset but a learned layer for encoding time reduction confined to the intra-frame partition search, leaving every other encoding stage untouched. The main strategy is to remove candidates progressively as stronger evidence becomes available: the first stage decides how much of the tree deserves to be opened at all, from causal context whose extraction cost is negligible, and the second whether the most expensive family of candidates is worth evaluating, from the rate-distortion evidence the encoder has by then measured. Over the eight 4K sequences of the Common Test Conditions (CTC) Class A1, SNP-AV1 reaches

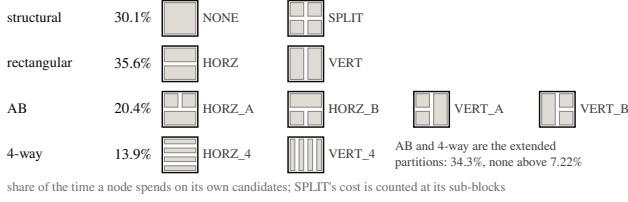


Fig. 1. The ten partition shapes evaluated at a node, grouped into the four families used in this work and annotated with the share of local search time each family consumes.

an average time savings of 18.74% with a Bjøntegaard delta bitrate (BD-BR) of 0.586%.

II. BLOCK PARTITIONING IN AV1

The basic unit of the AV1 partitioning is the superblock, which has 128×128 or 64×64 pixels, and the structure that subdivides it is called partition tree [5]. AV1 allows ten partition types at each node [6], which Fig. 1 organizes into the four families used throughout this paper: the two structural decisions, PARTITION_NONE, hereafter NONE, which codes the node undivided, and PARTITION_SPLIT, the quad-tree split into four square sub-blocks; the two rectangular shapes, PARTITION_HORZ and PARTITION_VERT; the four asymmetric AB shapes, HORZ_A, HORZ_B, VERT_A and VERT_B, combining one rectangular half with two square sub-blocks; and the two 4-way shapes, HORZ_4 and VERT_4, of 4:1 aspect ratio. The AB and the 4-way families are jointly called the extended partitions.

The evaluation order inside a node is fixed and defines which information has already been paid at each point of the flow: NONE first, then the quad-tree split, then the rectangular shapes, and finally the extended partitions. A pruner acting before the first evaluation is blind to any rate-distortion number, and one acting after NONE has the measured rate, distortion and RD cost of the undivided candidate, at the price of never saving the cost of producing them.

The cost distribution in Fig. 1 was measured with the native partition timing instrumentation of libaom over 875,317 decision nodes of the 4K dataset described in Section III-B, at `cpu-used=0`, taking as local work the sum of the nine non-recursive candidates of a node. It splits into three blocks of comparable size, and the six extended shapes are one of them, at 34.3% of the local search time. No single extended shape exceeds 7.22% of that time, which is why the cost went unnoticed, and it concentrates in the large nodes, reaching 50.0% at 32×32 and 51.0% at 64×64 pixels, which are the nodes a 4K frame produces in abundance.

This work is focused on the recursive block partitioning decision of the AV1 intra-frame coding. The other decisions taken inside a coding block, such as the intra prediction modes, the transform type search and the in-loop filtering, lie outside the partition search loop attacked here, as does the inter-frame partition search, since the solution is trained and evaluated exclusively under the All-Intra (AI) encoder configuration.

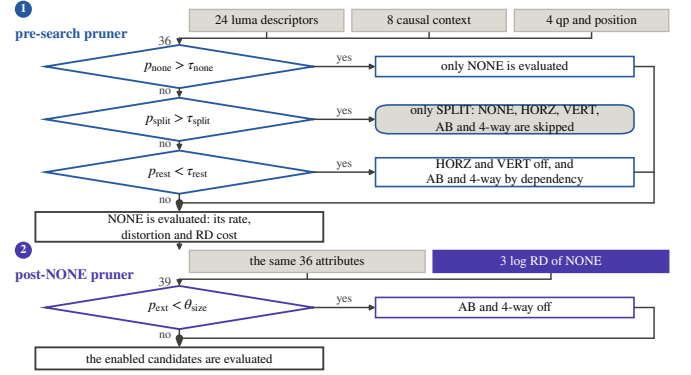


Fig. 2. Insertion points of the two pruners in the search of a node, and the cascade of conditions each of them applies.

III. PROPOSED SNP-AV1 SOLUTION

The Stacked Neural Pruners for AV1 (SNP-AV1) is a fast decision algorithm composed of two neural pruners inserted at two distinct stages of the search of a node, as shown in Fig. 2.

The two stages are complementary rather than competing. The first acts before any evaluation, so a correct commitment removes every competing candidate of the node and the whole subtree below it, and with that subtree the evaluation of NONE at every descendant, a cost no later pruner can save; it pays for that reach by deciding blind to rate-distortion. The second is better informed but limited in reach: it aims at the extended family, which the first reaches only by disabling the rectangular shapes, and it acts where the first did not commit, so the two cuts add up.

A. Attribute Design

The attribute vector of the first stage has thirty-six entries in three blocks. Twenty-four are luma descriptors of the node and of its hierarchical context, covering variance, gradient energy and orientation, row and column profiles, edge strength and density, and the contrast against the parent and the sibling sub-blocks. Eight carry the causal partitioning neighborhood: availability of the above and left neighbors, their decided widths and heights in log2 units, and its granularity and anisotropy. The last four carry the effective dequantization step of the luma DC coefficient, the normalized position in the frame and the node depth.

The criterion for inclusion was the cost of obtaining each attribute where the pruner is consulted: the twelve neighborhood, quantization and position entries are a free read of state the native search already maintains, and the luma descriptors require only arithmetic linear in the area of the node. The second stage reads these thirty-six plus three that exist only afterwards, the logarithm of the rate, of the distortion and of the RD cost measured for NONE. Every luma sample is brought to the eight-bit scale before any descriptor is computed, and the dequantization step with it, so no attribute depends on the bit depth of the source and models trained

on eight-bit material are deployed unchanged on the ten-bit evaluation sequences.

B. Dataset Construction

The reference software libaom v3.10.0 was instrumented under a compile-time guard to record, at every node of the intra-frame partition search, the decision taken by the encoder itself together with the luma samples, the causal context, the quantization state, the position and the rate-distortion result of NONE. The guard is necessary because a binary that writes to disk at every node cannot measure wall-clock acceleration; with it disabled, the build is byte-identical to the untouched reference tree.

The dataset was built from the sixteen 4K sequences of the UVG dataset [20], at 3840×2160 , 8-bit depth and 4:2:0 chroma subsampling, each encoded at four constant quality (CQ) values, 20, 32, 43 and 55, over five frames sampled uniformly along the whole clip, since in AI coding consecutive frames are nearly identical. The extraction ran entirely at `cpu-used=0`, because the label is only faithful if it comes from a search that evaluated every candidate shape, and any faster preset already prunes heuristically. This was verified: an extraction at `cpu-used=6` produced zero counts for the six extended classes, exactly the ones the second stage must predict.

The resulting dataset contains 26.98 million samples, of which 10.07 million are decision nodes of 16, 32 and 64 pixels; the rest are 8×8 leaves whose label is constant and which are left out of the modeling. It was partitioned by sequence into ten training, three validation and three reserved test sequences, frozen before any measurement, and the eight evaluation sequences belong to the CTC Class A1 set, absent from the UVG corpus. No sequence is therefore reused across training, testing and evaluation, which rules out data leakage.

C. Network Architecture and Training

Both stages use multilayer perceptrons, one per node size, trained independently for the 16×16 , 32×32 and 64×64 levels; the 8×8 level is terminal and is not modeled. The topology is common to all six networks and deliberately shallow: two hidden layers of 64 and 32 rectified units and a linear output whose softmax gives three class probabilities in the first stage and two in the second, for 27,759 parameters across the six deployed networks.

The networks were trained in PyTorch [21] with the AdamW optimizer [22], a learning rate of 1×10^{-3} , a weight decay of 1×10^{-4} , batches of 4096 samples and a fixed budget of 30 epochs without early stopping, under hard-label cross entropy without class weighting. The attributes are standardized during training and the standardization is folded into the first linear layer at export time, so the encoder feeds raw attributes, the models execute natively within the encoding pipeline and no new inference code in C is required; this parity was verified over 192 random vectors, with a maximum difference of 1.35×10^{-7} .

D. Pruning Policies and Encoder Integration

The first-stage pruner applies three actions in cascade, drawn in Fig. 2, on the probabilities p_{none} , p_{split} and p_{rest} of the undivided, split and remaining classes. If $p_{\text{none}} > \tau_{\text{none}}$, the node is decided as NONE and the whole subtree is cut without evaluating any other candidate; this is the primary lever of time savings. Otherwise, if $p_{\text{split}} > \tau_{\text{split}}$, NONE is skipped as well and the search recurses only into the four sub-blocks. Otherwise, if $p_{\text{rest}} < \tau_{\text{rest}}$, the rectangular candidates are disabled, and the extended ones with them by dependency; otherwise the node proceeds to the full search. Two operating points were frozen before the final campaign: the balanced one uses $\tau_{\text{none}} = 0.95$, $\tau_{\text{split}} = 0.90$ and $\tau_{\text{rest}} = 0.20$, the aggressive one 0.60, 0.85 and 0.40.

The second-stage pruner marks the node to skip the extended partitions when p_{ext} , the estimated probability that an AB or a 4-way shape wins the node, falls below a threshold θ kept per node size, leaving NONE, the quad-tree split and the rectangular shapes untouched. The per-level choice is experimental, since the values that discard the same fraction of true winners differ widely: calibrated on the reserved sequences to lose a tenth of the nodes whose optimal shape is extended, the deployed thresholds are 0.0910, 0.1031 and 0.0144 at the 16, 32 and 64-pixel levels, and a stronger calibration gives 0.1776, 0.1627 and 0.0321.

Both pruners were implemented in C inside libaom v3.10.0. The policy only removes candidates and never forces an illegal one, so SNP-AV1 is fully compliant with the AV1 specification and does not affect the bitstream syntax.

IV. RESULTS AND COMPARISONS

The developed solution was evaluated using the libaom v3.10.0 AV1 reference software, following the AOM Common Test Conditions (CTC) [23] and under the AI encoder configuration. The eight 4K sequences of the CTC Class A1 test set, named in Table I, are all at 3840×2160 , 10 bits and 4:2:0 [24]. The first 15 frames of each were encoded, as the CTC specifies for the AI scenario, at CQ 20, 32, 43 and 55, against the unmodified libaom at `cpu-used=0` as anchor. The BD-BR [25] is computed over the luma peak signal-to-noise ratio, and all values reported here are positive, hence coding efficiency losses. Writing $t_{\text{cfg}}(s, q)$ and $t_{\text{anc}}(s, q)$ for the encoding times of the evaluated configuration and of the anchor on sequence s at quality point q , over the sequence set S and the quality set Q ,

$$\text{TS} = \frac{1}{|S|} \sum_{s \in S} \frac{1}{|Q|} \sum_{q \in Q} \left(1 - \frac{t_{\text{cfg}}(s, q)}{t_{\text{anc}}(s, q)} \right), \quad (1)$$

reported as a percentage. Taking the ratio per quality point before any average keeps the four points equally weighted. All BD-BR and TS results come from this grid; the shares in Fig. 1 were measured on the UVG corpus described in Section III-B instead.

Every configuration was encoded once over this grid. Repeating the anchor, the first stage at the balanced thresholds

TABLE I
PER-SEQUENCE RESULTS OF THE TWO DEPLOYED OPERATING POINTS
AGAINST THE LIBAOM V3.10.0 ANCHOR AT `cpu-used=0`

Sequence	Balanced		Aggressive	
	BD-BR	TS	BD-BR	TS
BoxingPractice	0.662%	20.33%	1.560%	31.97%
Crosswalk	0.534%	20.53%	1.145%	35.08%
FoodMarket2	0.609%	14.14%	1.386%	26.74%
Neon1224	0.685%	22.00%	1.427%	33.05%
NocturneDance	0.168%	13.78%	1.209%	30.79%
PierSeaSide	0.458%	17.47%	1.403%	26.03%
Tango	1.139%	22.89%	2.206%	38.80%
TimeLapse	0.433%	18.78%	0.934%	31.00%
Average	0.586%	18.74%	1.409%	31.68%

and `cpu-used=1` five times on Crosswalk gives a standard deviation of 0.23 percentage points on TS, the resolution against which the differences below are to be read.

Fig. 3 places the measured operating space of SNP-AV1 against the anchor and against the native speed presets, and it must be qualified before any number is read, because the two arms do not have the same scope. A preset is a global switch: moving from `cpu-used=0` to 1 turns on the native learned partition pruner of libaom, off at the anchor, and with it dozens of heuristics spread over every other coding stage. SNP-AV1 acts inside the partition search alone. The preset curve therefore answers whether the speed knob already suffices, and does not rank partition-search techniques.

That native pruner is architecturally distinct as well. It reads a luma window and the quantization step through a convolutional network shared by the whole superblock, without causal partitioning context; it is consulted before the search only, so it never observes measured rate-distortion; and its action is binary on the square axis, so the extended partitions are reached only by dependency. The second pruner of SNP-AV1 has therefore no counterpart in it.

Under that reading, the balanced calibration delivers time savings of 17.72%, 18.74% and 19.81% at a BD-BR of 0.568%, 0.586% and 0.651%, and the aggressive one 31.51%, 31.68% and 32.16% at 1.403%, 1.409% and 1.420%; inside each family the three points differ only in how much of the extended family the second pruner is allowed to remove. The second stage is measured marginally, because it acts only on the residue of nodes the first stage did not commit. Over the balanced base it contributes 1.02 percentage points of time savings for 0.018 of BD-BR, buying time about 3.5 times cheaper than the only alternative available, loosening the thresholds of the first stage; over the aggressive base it falls below the measurement resolution, because a loosened undivided threshold commits the nodes before they ever reach the second pruner.

Table I opens the two deployed points per sequence, and what it exposes is content dispersion, not a swept range. Read as time bought per point of coding efficiency ceded, NocturneDance is the most favourable sequence and Tango the least, with a median near 35 points of time savings per

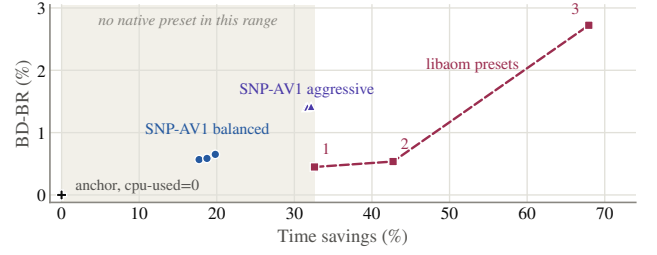


Fig. 3. Operating space over the CTC Class A1 4K sequences: the six SNP-AV1 operating points and the native preset ladder, against the libaom v3.10.0 anchor at `cpu-used=0`.

point of BD-BR, and that ordering is stable across the two calibrations.

Under the same reading, the ladder measures 32.59% of time savings at a BD-BR of 0.449% for `cpu-used=1`, so its first rung alone already dominates every point of SNP-AV1 on both axes. What the ladder does not offer is granularity: it jumps from the anchor straight to roughly 32.6%, and the six measured points of SNP-AV1 populate that void, three of them between 17.72% and 19.81%, obtained by threshold alone, without retraining or recompiling.

The overhead is reported rather than omitted: the complete path of SNP-AV1, attribute extraction and inference together, and that of the native learned pruner both stay below one third of one percent of the encoding time, so the time savings come from the pruning decisions and not from the lightness of each inference. This operating region is nevertheless reached from a modest budget: six deliberately shallow networks of 27,759 parameters, a fixed schedule of 30 epochs, and a model selection limited to a few ablations run on the validation split and never on the test sequences. Architecture and schedule were therefore barely explored, which is itself the opportunity.

V. CONCLUSIONS

This paper presented SNP-AV1, a machine-learning solution that stacks two neural pruners at two distinct stages of the AV1 intra-frame partition search, reaching an average time savings of 18.74% with a BD-BR of 0.59%, and 32.16% with 1.42% under a stronger calibration. The first pruner decides how much of the subtree is opened before any rate-distortion cost is paid and the second whether the extended partitions are evaluated; measured marginally over the first, the second adds 1.02 percentage points of time savings for 0.018 of BD-BR. No operating point dominates the native preset ladder on both axes, but that ladder reconfigures the whole encoder while SNP-AV1 acts only inside the partition search, at a granularity the presets do not offer.

ACKNOWLEDGMENT

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001, and also by the Brazilian research support agencies CNPq and FAPERGS.

REFERENCES

- [1] Statista. (2025). *Share of Internet Users Watching Online Videos Every Month From 1st Quarter 2022 to 3rd Quarter 2024*. [Online]. Available: <https://www.statista.com/statistics/1489445/online-video-viewers-worldwide-quarterly/>
- [2] ITU-T. (2013). *H.265: High Efficiency Video Coding*. [Online]. Available: <https://www.itu.int/rec/t-rec-h.265>
- [3] ITU-T. (2020). *H.266: Versatile Video Coding*. [Online]. Available: <https://www.itu.int/rec/t-rec-h.266>
- [4] Alliance for Open Media. (2019). *AV1 Video Codec*. [Online]. Available: <https://aomedia.org/av1>
- [5] J. Han *et al.*, “A technical overview of AV1,” *Proc. IEEE*, vol. 109, no. 9, pp. 1435–1462, Sep. 2021.
- [6] Y. Chen *et al.*, “An overview of core coding tools in the AV1 video codec,” in *Proc. Picture Coding Symp. (PCS)*, Jun. 2018, pp. 41–45.
- [7] D. Grois, T. Nguyen, and D. Marpe, “Coding efficiency comparison of AV1/VP9, H.265/MPEG-HEVC, and H.264/MPEG-AVC encoders,” in *Proc. Picture Coding Symp. (PCS)*, Dec. 2016, pp. 1–5.
- [8] M. A. Layek *et al.*, “Performance analysis of H.264, H.265, VP9 and AV1 video encoders,” in *Proc. 19th Asia-Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sep. 2017, pp. 322–325.
- [9] G. J. Sullivan and T. Wiegand, “Rate-distortion optimization for video compression,” *IEEE Signal Process. Mag.*, vol. 15, no. 6, pp. 74–90, Nov. 1998.
- [10] Alliance for Open Media. *AV1 Codec Library*, v3.10.0. [Online]. Available: <https://aomedia.googlesource.com/aom>
- [11] M. Xu and B. Jeon, “Selection of intra prediction tools for fast AV1 encoding,” in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Oct. 2020, pp. 1–4.
- [12] M. Xu and B. Jeon, “User-priority based AV1 coding tool selection,” *IEEE Trans. Broadcast.*, vol. 67, no. 3, pp. 736–745, Sep. 2021.
- [13] J. Jeong, G. Gankhuyag, and Y.-H. Kim, “A fast intra mode decision based on accuracy of rate distortion model for AV1 intra encoding,” in *Proc. 34th Int. Tech. Conf. Circuits/Syst., Comput. Commun. (ITC-CSCC)*, Jun. 2019, pp. 1–3.
- [14] M. Corrêa, B. Zatt, D. Palomino, G. Corrêa, and L. Agostini, “A high-throughput hardware architecture for AV1 non-directional intra modes,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 5, pp. 1481–1494, May 2020.
- [15] L. Netto, M. Corrêa, D. Palomino, L. Agostini, and G. Corrêa, “Power-quality configurable hardware design for AV1 directional intra-frame prediction,” *IEEE Design Test*, vol. 39, no. 2, pp. 38–45, Apr. 2022.
- [16] J. Kim, S. Blasi, A. S. Dias, M. Mrak, and E. Izquierdo, “Fast inter-prediction based on decision trees for AV1 encoding,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 1627–1631.
- [17] M. Saldanha, G. Sanchez, C. Marcon, and L. Agostini, “Configurable fast block partitioning for VVC intra coding using light gradient boosting machine,” *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 6, pp. 3947–3960, Jun. 2022.
- [18] Q. He, W. Wu, L. Luo, C. Zhu, and H. Guo, “Random forest based fast CU partition for VVC intra coding,” in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Aug. 2021, pp. 1–4.
- [19] J. Chen, H. Sun, J. Katto, X. Zeng, and Y. Fan, “Fast QTMT partition decision algorithm in VVC intra coding based on variance and gradient,” in *Proc. IEEE Vis. Commun. Image Process. (VCIP)*, Dec. 2019, pp. 1–4.
- [20] A. Mercat, M. Viitanen, and J. Vanne, “UVG dataset: 50/120fps 4K sequences for video codec analysis and development,” in *Proc. 11th ACM Multimedia Syst. Conf. (MMSys)*, Jun. 2020, pp. 297–302.
- [21] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Dec. 2019, pp. 8024–8035.
- [22] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2019, pp. 1–18.
- [23] X. Zhao *et al.*, “AOM common test conditions v9.0,” Alliance for Open Media, Codec Working Group, doc. CWG-G082, Mar. 2026.
- [24] Xiph.Org Foundation. *AOM CTC Test Set, Class A1 (4K)*. [Online]. Available: https://media.xiph.org/video/aomctc/test_set/a1_4k/
- [25] G. Bjøntegaard, “Calculation of average PSNR differences between RD-curves,” Video Coding Experts Group (VCEG), doc. VCEG-M33, Mar. 2001.